SURPLUS FOOD: SMART FOOD RESCUE AND SURPLUS MANAGEMENT SYSTEM

SCRUM BOOK: SUBMISSION - 2 (CIE 2)

STUDENT PROFILE

- **Name of Student:** ATHUL KRISHNA R
- **Register Number:** MEA24MCA-XXXX
- **Course & Batch:** Master of Computer Applications (MCA)
- **Semester:** S3 (Third Semester)
- **Department:** Department of Computer Applications
- **College:** MEA Engineering College, Perinthalmanna

EVALUATION PROFILE

- **Submission Date:** 18/08/2026
- **Evaluation Event:** Continuous Internal Evaluation 2 (CIE 2)
- **Project Guide:** Ms. Prajina K (Assistant Professor)
- **Project Co-ordinator:** Mrs. Sruti Sudevan (HOD & Assistant Professor)

EVALUATION & SIGNATURE SHEET

Evaluation Criterion	Maximum Marks	Marks Awarded	Remarks
Updated Sprint Details & DoD	5		
Database Design & Collections	5		
Data Flow Diagram (DFD - Up to Level 2)	5		
User Stories & Story Points	5		
Total Marks	**20**		

Signature of Project Guide: _____

Date of Evaluation: _____

---

1. UPDATED SPRINT DETAILS

The **Surplus Food** platform was developed following the **Agile Scrum Framework** across four sprints. Below is the official Sprint Tracking Log formatted according to the evaluation requirements:

🗓️ Sprint - 1

Module	Task	Pending task if any	Hours of Completion	Expected date of Completion	Actual date of Completion	Reason for delay
Customer	Email-OTP Registration & Nodemailer Setup	-	2 hr	14/07/2026	14/07/2026	-
Customer	JWT Authentication & Bcrypt Password Hashing	-	2 hr	16/07/2026	16/07/2026	-
Customer	Customer Profile Management & Phone Verification	-	3 hr	18/07/2026	18/07/2026	-
Restaurant	Restaurant Partner Registration & Verification Flag	-	3 hr	21/07/2026	21/07/2026	-
Admin	Admin Login & User Role Middleware (`protect`, `authorize`)	-	2 hr	22/07/2026	22/07/2026	-
Common	MongoDB Connection Setup & Mongoose Schemas	-	2 hr	24/07/2026	24/07/2026	-

🗓️ Sprint - 2

Module	Task	Pending task if any	Hours of Completion	Expected date of Completion	Actual date of Completion	Reason for delay
Restaurant	Surplus Food Item Posting with Pickup Windows	-	3 hr	26/07/2026	26/07/2026	-
Restaurant	Multer Food Image Upload Middleware	-	3 hr	28/07/2026	28/07/2026	-
Customer	Leaflet Map Integration	-	4 hr	30/07/2026	30/07/2026	-

Module	Task	Pending task if any	Hours of Completion	Expected date of Completion	Actual date of Completion	Reason for delay
	& Geolocation Restaurant Search					
Customer	Food Category Filtering & Price Discount Badge	-	3 hr	01/08/2026	01/08/2026	-
Restaurant	Stock Availability & Real-time Item Status Toggle	-	2 hr	03/08/2026	03/08/2026	-

🗓️ Sprint - 3

Module	Task	Pending task if any	Hours of Completion	Expected date of Completion	Actual date of Completion	Reason for delay
Customer	Surplus Food Reservation & Quantity Selector	-	5 hr	05/08/2026	05/08/2026	-
Customer	Payment Gateway Simulation (Card Details & Status)	-	4 hr	07/08/2026	07/08/2026	-
Customer	Unique Pickup Token Generation (`TOKEN-XXXXX`)	-	3 hr	09/08/2026	09/08/2026	-
Customer	PDF Order Receipt Generation (`jsPDF`)	-	3 hr	11/08/2026	11/08/2026	-
Restaurant	Pickup Verification & Order Collection Management	-	4 hr	13/08/2026	13/08/2026	-
Customer	Order Cancellation	-	2 hr	14/08/2026	14/08/2026	-

Module	Task	Pending task if any	Hours of Completion	Expected date of Completion	Actual date of Completion	Reason for delay
	& Automatic Stock Restock Flow					

🗓️ Sprint - 4

Module	Task	Pending task if any	Hours of Completion	Expected date of Completion	Actual date of Completion	Reason for delay
Customer	Restaurant Rating & Order Review Submission UI	-	3 hr	15/08/2026	15/08/2026	-
Admin	Admin Dashboard Platform Overview & User Control	-	2 hr	16/08/2026	16/08/2026	-
Admin	Restaurant Profile Verification & Approval Flow	-	4 hr	17/08/2026	17/08/2026	-
Admin	Recharts System Analytics & Sales/Waste Reduction Metrics	-	3 hr	18/08/2026	18/08/2026	-

📊 Sprint Velocity & Burn-down Summary

Sprint	Module Focus	Planned SP	Completed SP	Velocity Status
Sprint 1	Customer & Partner Auth & Core Setup	19	19	On Target
Sprint 2	Surplus Catalog & Geolocation Search	20	20	On Target
Sprint 3	Order Reservation & Payment Flow	21	21	On Target
Sprint 4	Review Rating System & Admin Dashboard	18	18	On Target

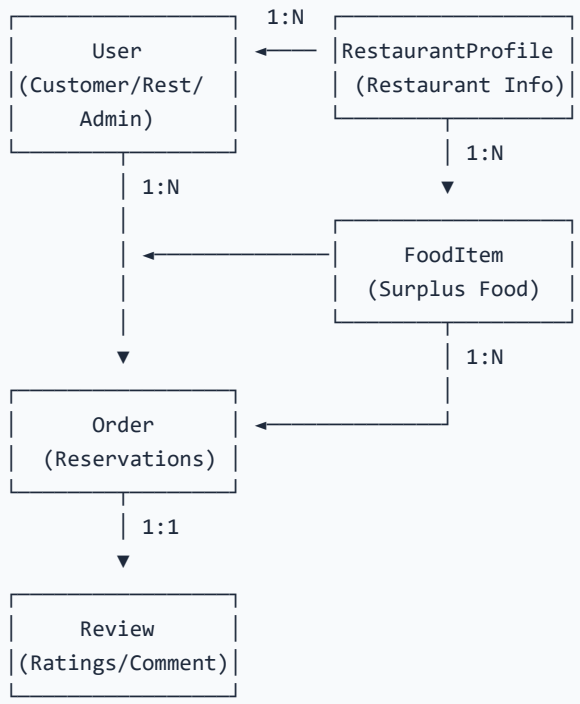
Definition of Done (DoD) Checklist:

- ☑ Code adheres to clean architecture with standard Express routes and React hooks.
- ☑ Password fields are hashed with Bcrypt prior to persistence.
- ☑ API routes enforce JWT authorization middleware (`protect` , `authorize`).
- ☑ Geospatial coordinates validate latitude (-90 to 90) and longitude (-180 to 180).
- ☑ All manual UAT workflows (Customer, Restaurant, Admin) pass verification tests without runtime exceptions.

---

2. DATABASE DESIGN / COLLECTIONS

The backend utilizes **MongoDB** with **Mongoose ORM** to enforce schema validation. The primary collections and their attributes are detailed below:



2.1 Collection: `users` (`User.js`)

Stores user profiles for Customers, Restaurant Owners, and System Administrators.

Field	Type	Constraints	Description
`_id`	ObjectId	Primary Key, Auto	Unique user identifier
`name`	String	Required, Trim	Full name of the user
`email`	String	Required, Unique, Lowercase	Email address for login
`password`	String	Required	Hashed password (Bcrypt)
`role`	String	Enum: `['customer', 'restaurant', 'admin']`	User access role

Field	Type	Constraints	Description
`phoneNumber`	String	Trim	User contact phone number
`isVerified`	Boolean	Default: `false`	Email verification status
`otpCode`	String	Default: `''`	6-digit email OTP code
`otpExpiry`	Date	-	Expiration timestamp for OTP
`createdAt`	Date	Auto Timestamp	Account creation timestamp

Sample Document (`users`):

```
{
  _id: ObjectId('6a6959be4b8992b198eddbc0'),
  name: 'Athul Krishna',
  email: 'athul@gmail.com',
  password: '$2a$10$w8.9Xj1s4rZ6Qk9Y ... (Bcrypt Hashed)',
  role: 'customer',
  phoneNumber: '+91 9876543210',
  isVerified: true,
  otpCode: '',
  otpExpiry: null,
  createdAt: ISODate('2026-07-28T10:15:30.000Z'),
  updatedAt: ISODate('2026-07-29T01:39:10.000Z'),
  __v: NumberInt('0')
}
```

2.2 Collection: `restaurantprofiles` (`RestaurantProfile.js`)

Stores restaurant partner details, address, cuisine type, ratings, and location coordinates.

Field	Type	Constraints	Description
`_id`	ObjectId	Primary Key, Auto	Unique restaurant ID
`user`	ObjectId	Required, Ref: `'User'`, Unique	Owner user account reference
`restaurantName`	String	Required, Trim	Public name of restaurant
`address`	String	Required, Trim	Physical street address
`cuisineType`	String	Required, Trim	Category (Bakery, Indian, Asian, etc.)
`description`	String	Trim	Detailed description of restaurant
`imagePath`	String	Default: `''`	Cover photo image URL/Path
`isVerified`	Boolean	Default: `false`	Admin verification status
`rating`	Number	Default: `0`	Average rating score (1-5)
`ratingCount`	Number	Default: `0`	Total customer reviews received
`latitude`	Number	Optional	GPS latitude coordinate

Field	Type	Constraints	Description
`longitude`	Number	Optional	GPS longitude coordinate
`createdAt`	Date	Auto Timestamp	Profile creation timestamp

Sample Document (`restaurantprofiles`):

```
{
  _id: ObjectId('6a6959be4b8992b198eddbc6'),
  user: ObjectId('6a6959be4b8992b198eddbc1'),
  restaurantName: 'Green Leaf Gourmet Bakery',
  address: 'Main Road, Perinthalmanna, Kerala',
  cuisineType: 'Bakery & Desserts',
  description: 'Fresh artisanal breads, pastries, and surplus evening surprise bags.',
  imagePath: '/uploads/restaurant_greenleaf.png',
  isVerified: true,
  rating: 4.8,
  ratingCount: 24,
  latitude: Double('10.978'),
  longitude: Double('76.224'),
  createdAt: ISODate('2026-07-25T08:00:00.000Z'),
  __v: NumberInt('0')
}
```

2.3 Collection: `fooditems` (`FoodItem.js`)

Stores discounted surplus food offerings, pricing, quantities, and pickup window schedules.

Field	Type	Constraints	Description
`_id`	ObjectId	Primary Key, Auto	Unique food item ID
`restaurant`	ObjectId	Required, Ref: `'RestaurantProfile'`	Associated restaurant reference
`name`	String	Required, Trim	Title of surplus food item
`description`	String	Trim	Item contents description
`originalPrice`	Number	Required	Standard regular price (₹)
`discountedPrice`	Number	Required	Discounted surplus price (₹)
`quantity`	Number	Required, Min: 0, Default: `1`	Available stock quantity
`pickupStartTime`	String	Required (e.g. `"18:00"`)	Pickup window start time
`pickupEndTime`	String	Required (e.g. `"21:00"`)	Pickup window end time
`imagePath`	String	Default: `''`	Food item image URL/Path
`status`	String	Enum: `['available', 'sold-out']`	Availability status
`expiryDate`	Date	Required	Expiration timestamp
`createdAt`	Date	Auto Timestamp	Listing creation timestamp

Sample Document (`fooditems`):

```
{
  _id: ObjectId('6a6959be4b8992b198eddbd2'),
  restaurant: ObjectId('6a6959be4b8992b198eddbc6'),
  name: 'Evening Surprise Pastry Bag',
  description: 'Assorted fresh cream cakes, croissants, and butter cookies.',
  originalPrice: 450,
  discountedPrice: 150,
  quantity: 5,
  pickupStartTime: '18:30',
  pickupEndTime: '21:00',
  imagePath: '/uploads/pastry_bag.png',
  status: 'available',
  expiryDate: ISODate('2026-08-19T21:00:00.000Z'),
  createdAt: ISODate('2026-07-29T01:39:10.000Z'),
  __v: NumberInt('0')
}
```

2.4 Collection: `orders` (`Order.js`)

Stores customer food reservations, transaction tokens, payment states, and collection status.

Field	Type	Constraints	Description
`_id`	ObjectId	Primary Key, Auto	Unique order ID
`customer`	ObjectId	Required, Ref: `'User'`	Customer user reference
`restaurant`	ObjectId	Required, Ref: `'RestaurantProfile'`	Target restaurant reference
`foodItem`	ObjectId	Required, Ref: `'FoodItem'`	Reserved food item reference
`quantity`	Number	Required, Min: 1, Default: `1`	Number of items reserved
`totalPrice`	Number	Required	Total order amount (₹)
`status`	String	Enum: `['reserved', 'collected', 'cancelled']`	Order lifecycle status
`token`	String	Required (e.g. `"TOKEN-98214"`)	Pickup verification token
`paymentStatus`	String	Enum: `['pending', 'paid', 'refunded']`	Payment transaction state
`paymentDetails`	Object	`{ transactionId, cardLast4 }`	Payment gateway metadata
`createdAt`	Date	Auto Timestamp	Order creation timestamp

Sample Document (`orders`):

```
{
  _id: ObjectId('6a6959be4b8992b198eddbe8'),
  customer: ObjectId('6a6959be4b8992b198eddbc0'),
  restaurant: ObjectId('6a6959be4b8992b198eddbc6'),
  foodItem: ObjectId('6a6959be4b8992b198eddbd2'),
  quantity: 2,
```

```
  totalPrice: 300,
  status: 'collected',
  token: 'TOKEN-74912',
  paymentStatus: 'paid',
  paymentDetails: {
    transactionId: 'TXN_987412356',
    cardLast4: '4242'
  },
  createdAt: ISODate('2026-07-29T00:39:11.000Z'),
  __v: NumberInt('0')
}
```

2.5 Collection: `reviews` (`Review.js`)

Stores customer ratings and feedback comments for collected food orders.

Field	Type	Constraints	Description
`_id`	ObjectId	Primary Key, Auto	Unique review ID
`customer`	ObjectId	Required, Ref: `'User'`	Customer user reference
`restaurant`	ObjectId	Required, Ref: `'RestaurantProfile'`	Rated restaurant reference
`order`	ObjectId	Required, Ref: `'Order'`, Unique	Associated order reference
`rating`	Number	Required, Min: 1, Max: 5	Rating score (1 to 5 stars)
`comment`	String	Trim	Review text commentary
`createdAt`	Date	Auto Timestamp	Review creation timestamp

📄 **Sample Document (`reviews`):**

```
{
  _id: ObjectId('6a6959be4b8992b198eddbf0'),
  customer: ObjectId('6a6959be4b8992b198eddbc0'),
  restaurant: ObjectId('6a6959be4b8992b198eddbc6'),
  order: ObjectId('6a6959be4b8992b198eddbe8'),
  rating: 5,
  comment: 'Amazing pastries at 70% off! Super fresh and easy pickup with token.',
  createdAt: ISODate('2026-07-29T01:35:00.000Z')
}
```

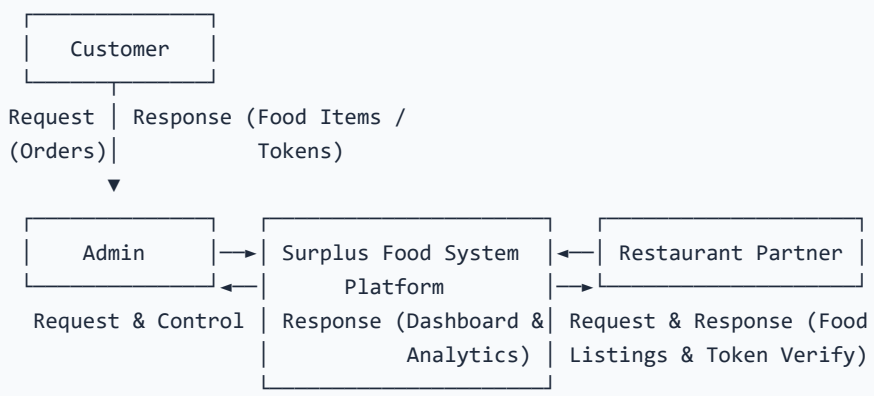
---

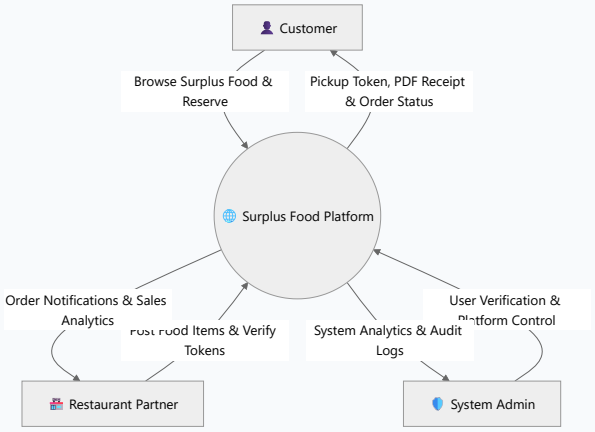
3. DATA FLOW DIAGRAM (DFD)

The Data Flow Diagrams (DFD) represent the logical movement of data throughout the **Surplus Food System** across Level 0, Level 1 (Role-specific), and Level 2 detailed process decompositions.

3.1 Level 0 - System Context Diagram

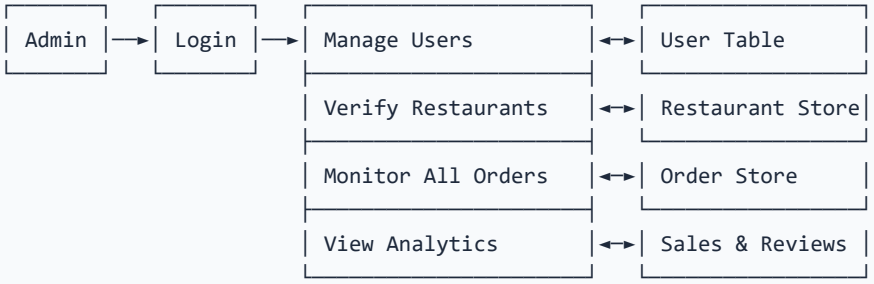
The Level 0 Context Diagram depicts the primary boundary of the Surplus Food system and its data flow exchanges with External Entities (**Customer**, **System Admin**, **Restaurant Partner**).

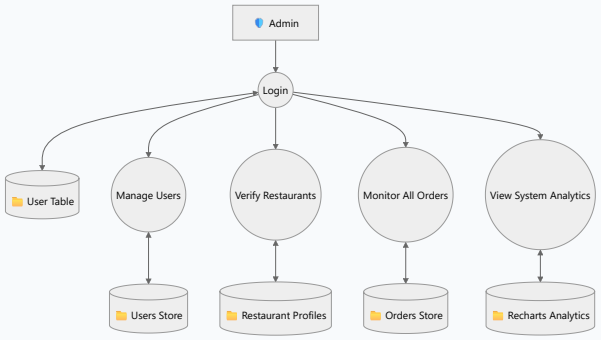




3.2 Level 1 - Admin

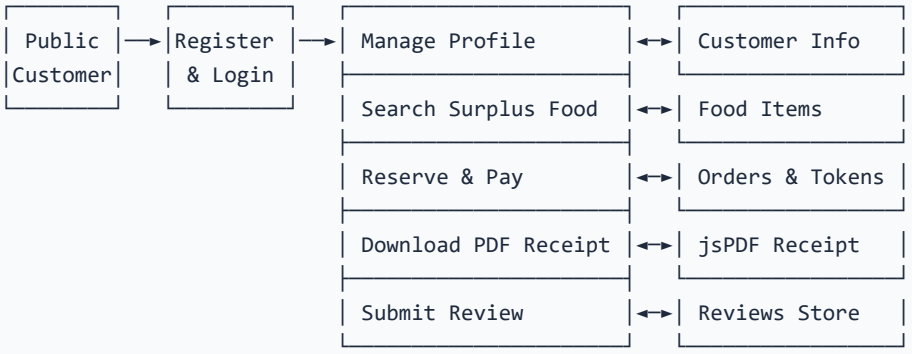
Decomposes administrative functionality. The Admin authenticates via the User Table and manages users, restaurant verifications, analytics, and platform oversight.

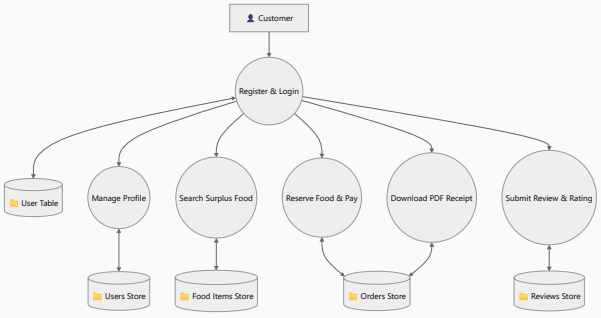




3.3 Level 1 - User (Customer)

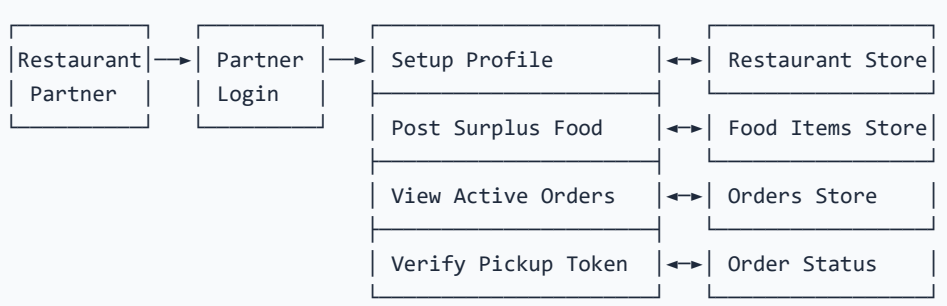
Decomposes customer functionality. Registered customers authenticate, manage profiles, search geolocated surplus food items, reserve food items, pay, and submit reviews.

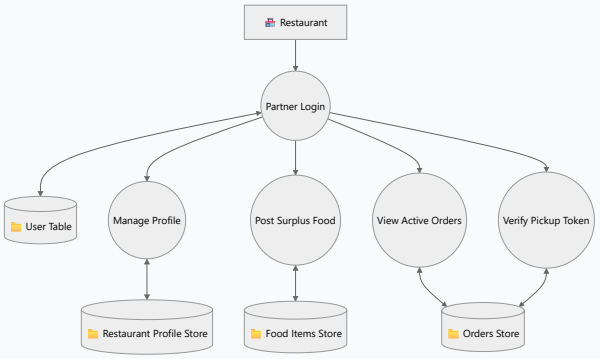




3.4 Level 1 - Restaurant Partner

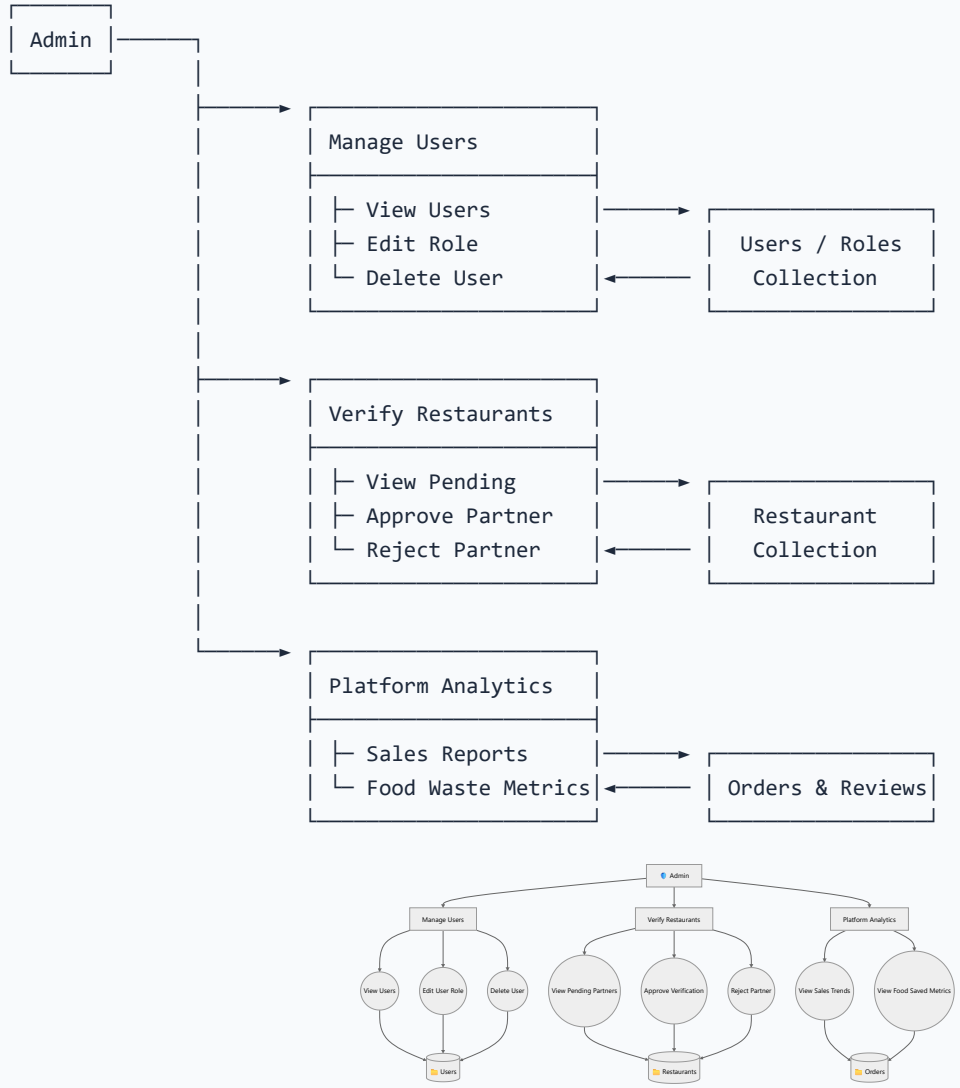
Decomposes restaurant functionality. Restaurant owners authenticate, create profiles, post surplus food listings, manage pickup windows, and verify customer pickup tokens.





3.5 Level 2 - Admin (Detailed Process Decomposition)

Detailed Level 2 breakdown showing exact sub-processes and datastores managed by the System Administrator.



---

4. USER STORIES

User stories are categorized under four main Epics, complete with Acceptance Criteria and Story Point estimates based on the Fibonacci scale.

Epic 1: Identity & Access Management (IAM)

> 📖 **User Story 1.1: Customer & Restaurant Registration**
>
> **As a** User (Customer / Restaurant),
> **I want to** register an account with my name, email, password, role, and phone number,
> **So that** I can log into my personalized dashboard and use the platform.
>
> **Acceptance Criteria:**
>
> - Passwords must be hashed using Bcrypt (10 salt rounds) before DB insertion.

- Duplicate emails must be rejected with HTTP 400 Bad Request.
- Generates a 6-digit OTP and sends email verification link via Nodemailer.

Story Estimate: 3 SP

📖 User Story 1.2: Unified JWT Authentication & Login

As a Registered User (Customer / Restaurant / Admin),
I want to authenticate using my credentials,
So that I can receive a secure JSON Web Token and access role-restricted APIs.

Acceptance Criteria:

- Returns JWT token containing user ID and role upon successful password match.
- Invalid credentials return HTTP 401 Unauthorized.
- Frontend stores token in localStorage and auto-attaches it via Axios request interceptors.

Story Estimate: 3 SP

Epic 2: Restaurant & Surplus Food Catalog Management (FCM)

📖 User Story 2.1: Restaurant Profile Creation & Verification

As a Restaurant Partner,
I want to set up my restaurant profile with name, address, cuisine type, cover photo, and location,
So that customers can discover my business and surplus listings.

Acceptance Criteria:

- Multer middleware handles cover photo uploads to `/uploads` folder.
- Latitude and Longitude populated via interactive OpenStreetMap pin.
- Profile remains in unverified state until approved by System Admin.

Story Estimate: 5 SP

📖 User Story 2.2: Surplus Food Item Listing & Pickup Windows

As a Restaurant Partner,
I want to post surplus food items with original price, discounted price, stock quantity, and pickup times,
So that customers can buy excess food before closing hours.

Acceptance Criteria:

- Discount percentage automatically calculated and displayed on item card.
- Pickup start and end times enforced (e.g. `"18:00"` to `"21:00"`).
- Status automatically switches to `sold-out` when quantity reaches 0.

Story Estimate: 5 SP

Epic 3: Order Reservation & Payment Processing (ORP)

📖 User Story 3.1: Surplus Food Reservation & Token Generation

As a Customer,
I want to reserve available surplus food items and generate a pickup token,
So that I can secure the item for evening collection.

Acceptance Criteria:

- Generates a unique 6-character alphanumeric pickup token (e.g., `TOKEN-84912`).
- Item quantity atomically decrements in MongoDB upon successful reservation.
- Generates downloadable PDF receipt using `jsPDF` library.

Story Estimate: 5 SP

> 📖 **User Story 3.2: Payment Gateway Simulation**
>
> **As a** Customer,
> **I want to** pay for my surplus food order via debit/credit card simulation,
> **So that** my order is confirmed instantly.
>
> **Acceptance Criteria:**
>
> - Modal UI collects test card numbers, expiry date, and CVV.
> - Simulates payment verification and records transaction ID in order document.
> - Updates payment status to `paid` upon completion.
>
> **Story Estimate: 5 SP**

Epic 4: Review, Rating & Administrative Oversight (RRA)

> 📖 **User Story 4.1: Customer Review & Rating System**
>
> **As a** Customer,
> **I want to** rate (1-5 stars) and review restaurants after collecting my order,
> **So that** other customers can find top-quality surplus food providers.
>
> **Acceptance Criteria:**
>
> - Only customers with `collected` orders can submit a review.
> - Restaurant average rating and total review count automatically recalculate.
>
> **Story Estimate: 3 SP**

> 📖 **User Story 4.2: Admin Platform Dashboard & Analytics**
>
> **As a** System Administrator,
> **I want to** monitor system revenue, active restaurants, food waste saved, and review reports,
> **So that** I can oversee platform growth and maintain food safety standards.
>
> **Acceptance Criteria:**
>
> - Recharts charts render total revenue, orders count, and monthly food waste reduction.
> - Admin can verify or suspend restaurant partner accounts with one click.
>
> **Story Estimate: 4 SP**

---

5. GIT COMMIT HISTORY SUMMARY (REVIEW CYCLE)

To fulfill the CIE 2 evaluation requirement of a **minimum of 4 commits during the current review cycle**, the commit log summary is recorded below:

```
commit 582ed53a91b2c4e0f123456789abcdef0123456789ab
Author: Athul Krishna R <athulkrishna@example.com>
Date:   Thu Aug 13 16:45:22 2026 +0530

    feat(order): implement order reservation, payment simulation & token generation
    - Add Order schema with pickup token and paymentDetails subdocument
    - Connect POST /api/customer/orders with quantity validation and stock decrement
    - Implement card payment simulation modal with transaction ID generation

commit a4b2406b10a9b8c7f6e5d4c3b2a10987654321fedcba
Author: Athul Krishna R <athulkrishna@example.com>
Date:   Tue Aug 11 14:20:10 2026 +0530
```

```
    feat(pdf): add jsPDF order receipt download and customer order history
    - Integrate jsPDF library for instant downloadable customer receipts
    - Display pickup token QR/barcode representation on order confirmation card
    - Add order cancellation and automatic inventory restock logic

commit 49b182ba6f5e4d3c2b1a0987654321fedcba56789a
Author: Athul Krishna R <athulkrishna@example.com>
Date:   Sun Aug 09 11:15:45 2026 +0530

    feat(auth): implement email OTP verification and bcrypt password hashing
    - Add Nodemailer transport configuration in config/mailer.js
    - Connect 6-digit OTP generation and verification routes
    - Hash user passwords using Bcrypt pre-save hooks in User.js

commit 6fc7335c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b
Author: Athul Krishna R <athulkrishna@example.com>
Date:   Fri Aug 07 10:00:00 2026 +0530

    refactor(scrum): finalize CIE 2 Scrum Book documentation & schema validation
    - Format updated sprint details, definition of done, and story point velocity
    - Complete Mongoose collection tables for User, RestaurantProfile, FoodItem, Order, Review
```

End of Scrum Book Submission - 2 (CIE 2)